Глубокое обучение с подкреплением

Модуль 5: Unity ML-Agents

Курс Hugging Face Deep RL

Полное учебное издание на основе MDX в формате LaTeX

Исходный материал: курс Deep RL от Hugging Face.

Содержание

1 Введение в Unity ML-Agents **2**

2 Как устроен Unity ML-Agents? **3**
- 2.1 Что такое Unity ML-Agents? . . . 3
- 2.2 Шесть основных компонентов . . . 4
- 2.3 Внутри компонента обучения . . . 4

3 Среда SnowballTarget **6**
- 3.1 Цель агента . . . 7
- 3.2 Функция вознаграждения и проблема её проектирования . . . 7
- 3.3 Пространство наблюдений . . . 8
- 3.4 Пространство действий . . . 9

4 Среда Pyramids **10**
- 4.1 Функция вознаграждения . . . 10
- 4.2 Пространство наблюдений . . . 11
- 4.3 Пространство действий . . . 12

5 (Необязательно) Что такое любопытство в глубоком обучении с подкреплением? **13**
- 5.1 Две главные проблемы современного RL . . . 13
- 5.2 Так что же такое любопытство? . . . 14
- 5.3 Практическая часть . . . 16
- 5.4 Модуль 5: введение в ML-Agents . . . 17
- 5.5 Приступим к обучению агентов . . . 18

6 Проверочный тест **27**
- 6.1 Ответы на тест и пояснения . . . 28

7 Заключение **29**

8 Бонус: создание собственных сред с Unity и ML-Agents **30**
- 8.1 Шаг 1. Научись пользоваться Unity . . . 31
- 8.2 Шаг 2. Создай простейшую среду по руководству . . . 31
- 8.3 Шаг 3. Усложняй и совершенствуй среды . . . 31

1 Введение в Unity ML-Agents



Рис. 1: Обложка модуля курса, посвящённого Unity ML-Agents [оригинал]

Одна из сложностей обучения с подкреплением — **создание сред**. К счастью, для этого можно использовать игровые движки. Такие движки, как Unity, Godot и Unreal Engine, — программы для создания видеоигр. Они отлично подходят для разработки сред, поскольку предоставляют физические движки, средства двумерной и трёхмерной визуализации и многое другое.

Разработчики одного из этих движков, Unity, создали Unity ML-Agents Toolkit — дополнение к Unity, которое позволяет **использовать игровой движок для создания сред и обучения агентов**. Именно так в первом бонусном модуле мы учили Huggy ловить палку!

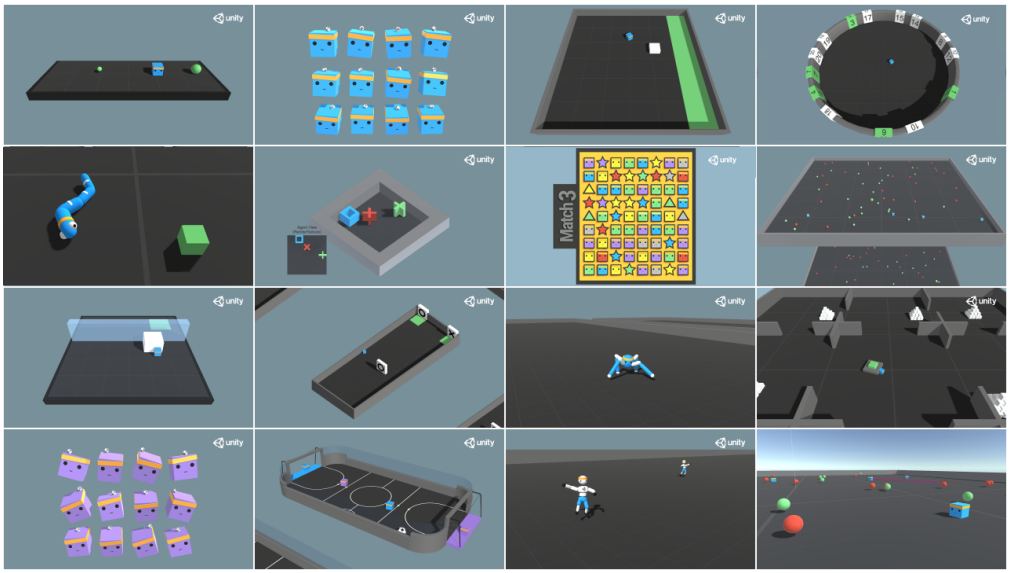


Рис. 2: Примеры сред из документации ML-Agents (источник: документация ML-Agents) [оригинал]

Unity ML-Agents Toolkit включает множество замечательных готовых сред: в них агенты играют в футбол, учатся ходить, перепрыгивают высокие стены и выполняют другие задачи.

В этом модуле мы научимся использовать ML-Agents, но **не переживай, если не умеешь работать в игровом движке Unity**: для обучения агентов осваивать сам редактор не потребуется.

Сегодня мы обучим двух агентов:

- Первый научится **бросать снежки в появляющиеся мишени**.
- Второму понадобится **нажать кнопку, чтобы появилась пирамида, добраться до неё, опрокинуть её, а затем подойти к золотому кирпичу на вершине**. Для этого агент должен исследовать среду; ему поможет метод, называемый любопытством (curiosity).

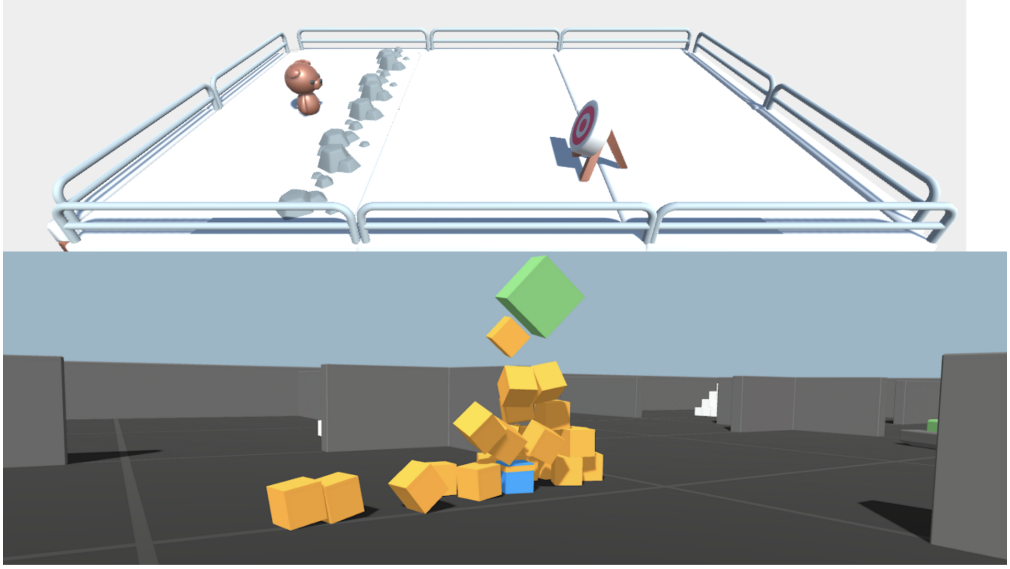


Рис. 3: Среды для обучения SnowballTarget и Pyramids [оригинал]

После обучения **мы загрузим агентов в Hugging Face Hub**. Ты сможешь **посмотреть на их поведение прямо в браузере, не запуская редактор Unity**.

Этот модуль **подготовит тебя к следующему испытанию: AI vs. AI, в котором предстоит обучать агентов в мультиагентных средах и соревноваться с агентами других участников курса**.

Звучит интересно? Тогда начнём!

2 Как устроен Unity ML-Agents?

Прежде чем обучать агента, разберёмся, **что такое ML-Agents и как он работает**.

2.1 Что такое Unity ML-Agents?

Unity ML-Agents — набор инструментов для игрового движка Unity, который **позволяет создавать собственные среды либо использовать готовые среды для обучения агентов**.

Его разрабатывает Unity Technologies — компания, создавшая один из самых известных игровых движков. На Unity, например, сделаны Firewatch, Cuphead и Cities: Skylines.



Рис. 4: Игра Firewatch создана на движке Unity [оригинал]

2.2 Шесть основных компонентов

В Unity ML-Agents есть шесть ключевых компонентов:

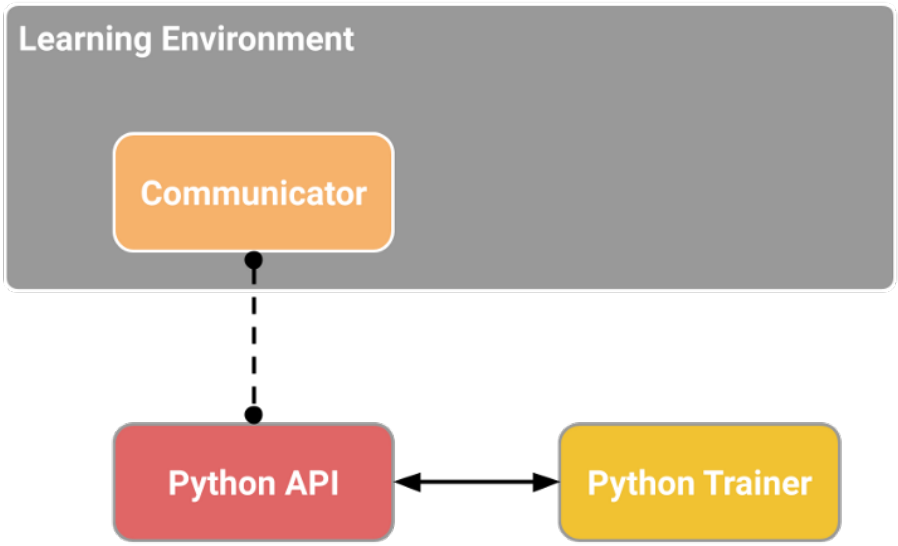


Рис. 5: Схема компонентов Unity ML-Agents (источник: документация Unity ML-Agents) [оригинал]

- Первый — *Learning Environment*, среда обучения. Она содержит **сцену Unity (среду) и её элементы**, например игровых персонажей.
- Второй — *Python Low-level API*, **низкоуровневый интерфейс Python для взаимодействия со средой и управления ею**. Через этот API мы запускаем обучение.
- Далее идёт *External Communicator* — компонент, который **связывает Learning Environment (на C#) с низкоуровневым Python API (на Python)**.
- *Python trainers* — **алгоритмы обучения с подкреплением, реализованные на PyTorch (PPO, SAC и др.)**.
- *Gym wrapper* — обёртка, представляющая среду обучения с подкреплением через интерфейс Gym.
- *PettingZoo wrapper* — обёртка на основе PettingZoo, мультиагентного аналога Gym.

2.3 Внутри компонента обучения

Внутри компонента обучения есть **два важных элемента**:

- Первый — *компонент агента*, действующего лица сцены. Мы будем **обучать агента, оптимизируя его политику** (она определяет, какое действие выбирать в каждом состоянии). В терминологии ML-Agents политика называется *Brain* («мозг»).
- Второй — *Academy* («академия»). Этот компонент **координирует агентов и процессы принятия ими решений**. Представь Academy в роли преподавателя, который обрабатывает запросы Python API.

Чтобы лучше понять её роль, вспомним цикл обучения с подкреплением. Его можно представить так:

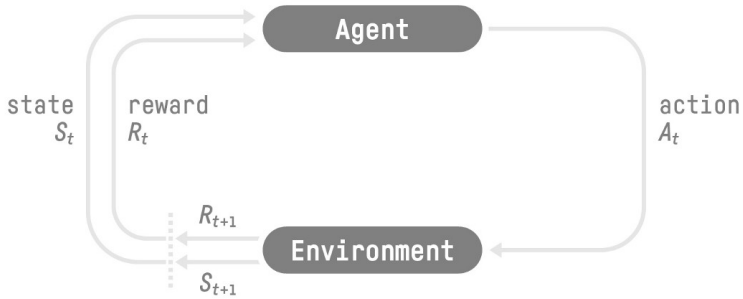


Рис. 6: Цикл RL: состояние, действие, вознаграждение и следующее состояние; источник: Ричард Саттон и Эндрю Барто, «Обучение с подкреплением: введение» [оригинал]

Представим агента, который учится играть в платформер. Цикл RL будет выглядеть следующим образом:

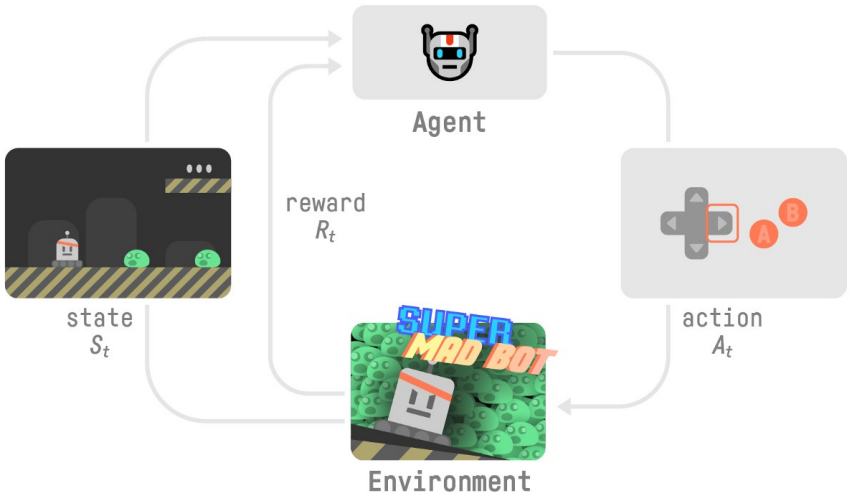


Рис. 7: Иллюстрация цикла обучения с подкреплением на примере игры [оригинал]

- Агент получает от **среды состояние** S_0 — например, первый кадр игры.
- На основе **состояния** S_0 агент выбирает **действие** A_0 — в данном случае перемещается вправо.
- Среда переходит в **новое состояние** S_1 — появляется следующий кадр.
- Среда выдаёт агенту **вознаграждение** R_1 — он не погиб, поэтому получает *положительное*

вознаграждение +1.

Такой цикл RL порождает последовательность **«состояние — действие — вознаграждение — следующее состояние»**. Цель агента — **максимизировать ожидаемое суммарное вознаграждение**.

Именно Academy будет **отдавать команды агентам и обеспечивать согласованность их работы**:

- Собирать наблюдения.
- Выбирать действие в соответствии с политикой.
- Выполнять действие.
- Сбрасывать среду после достижения предельного числа шагов или завершения эпизода.



Рис. 8: Компонент Academy в Unity ML-Agents и цикл принятия решений [оригинал]

Теперь мы понимаем, как устроен ML-Agents, и **готовы приступить к обучению агентов**.

3 Среда SnowballTarget

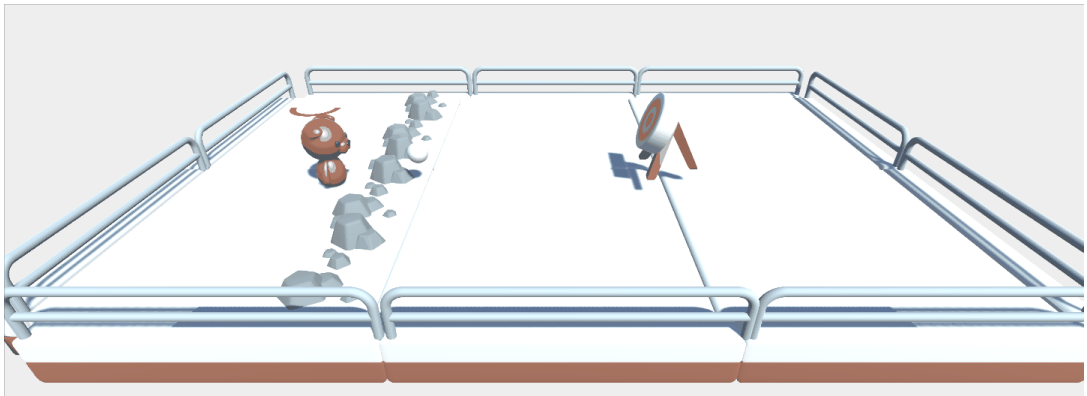


Рис. 9: Среда обучения SnowballTarget (кадр анимации) [оригинал]

SnowballTarget — среда, которую мы создали в Hugging Face, используя ресурсы Kay Lousberg. В конце модуля есть необязательный раздел **для тех, кто хочет научиться работать в Unity и создавать собственные среды**.

3.1 Цель агента

Первого агента, которого мы будем обучать, зовут медведь Жюльен (Julien). Его задача — **попадать снежками в мишени**.

Цель в этой среде — чтобы Жюльен **поразил как можно больше мишеней за ограниченное время** (1000 временных шагов). Для этого ему необходимо **правильно расположиться относительно мишени и выстрелить**.

Кроме того, чтобы избежать «спама снежками» (стрельбы на каждом временном шаге), **у Жюльена предусмотрен период восстановления**: после выстрела он должен подождать 0,5 секунды, прежде чем сможет стрелять снова.

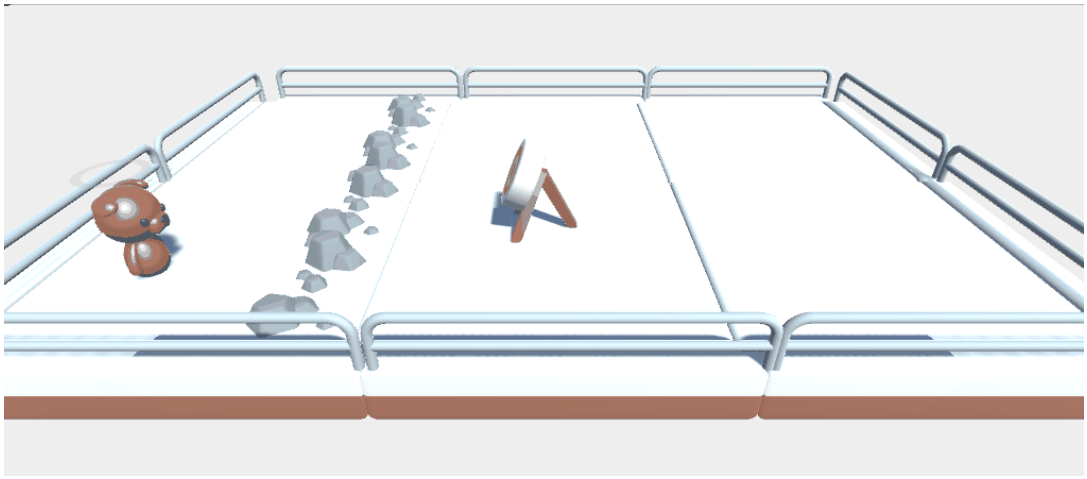


Рис. 10: После броска снежка агент должен подождать 0,5 секунды до следующего выстрела [оригинал]

3.2 Функция вознаграждения и проблема её проектирования

Функция вознаграждения здесь проста: **каждый раз, когда снежок агента попадает в мишень, среда выдаёт вознаграждение +1**. Поскольку цель агента — максимизировать ожидаемую накопленную награду, **он будет стремиться поразить как можно больше мишеней**.

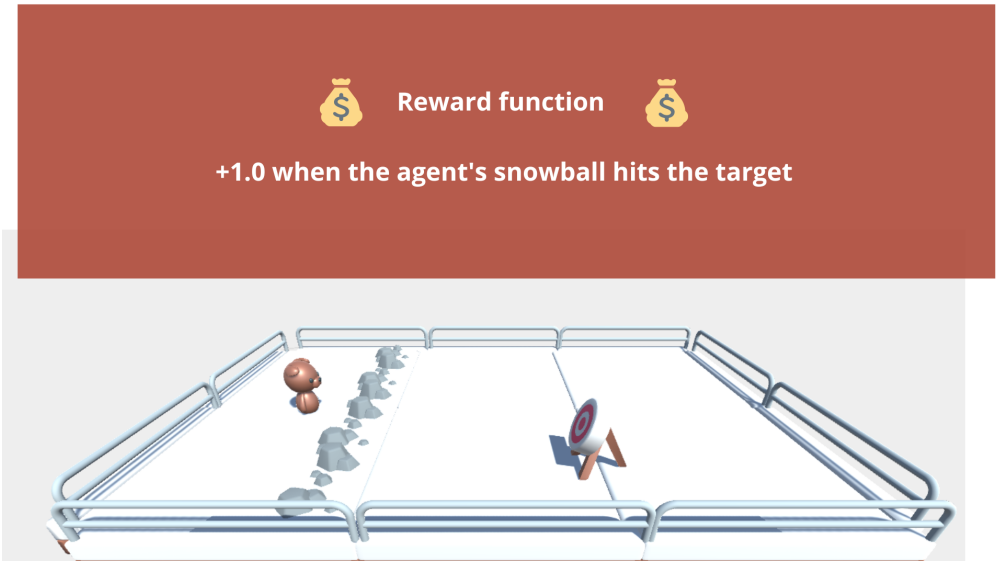


Рис. 11: Механизм выдачи вознаграждения в SnowballTarget [оригинал]

Можно было бы создать более сложную функцию вознаграждения, например добавить штраф, побуждающий агента действовать быстрее. Но при проектировании среды важно избегать *проблемы чрезмерного конструирования вознаграждения*: слишком сложной функции, которая на-

вязывает агенту желаемое поведение. Почему? Потому что **так можно упустить интересные стратегии, которые агент нашёл бы при более простой функции вознаграждения**.

В коде это выглядит так:

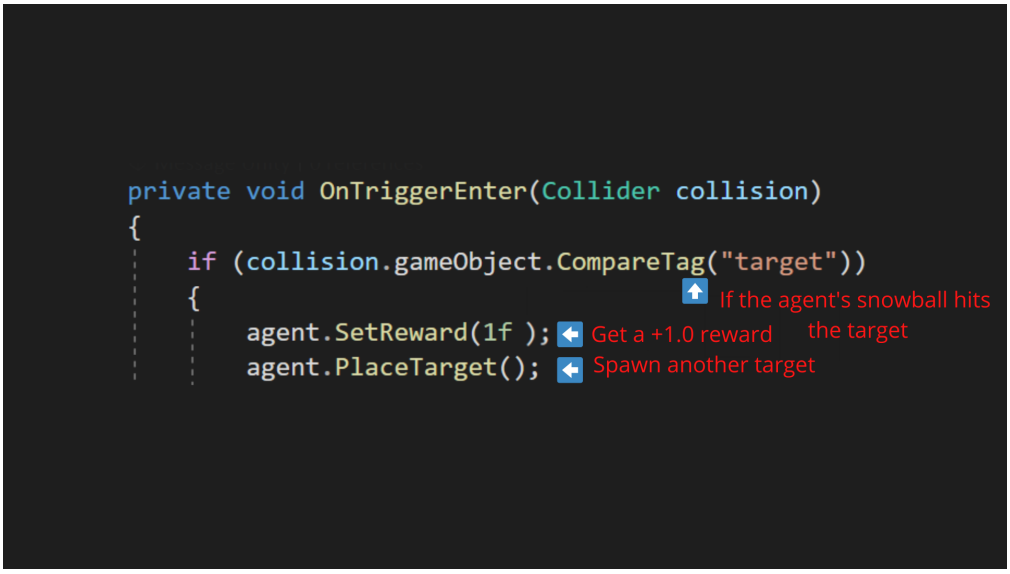


Рис. 12: Вычисление вознаграждения SnowballTarget в исходном коде [оригинал]

3.3 Пространство наблюдений

В качестве наблюдений здесь используется не обычное изображение (кадр), а **трассировка лучей (raycasts)**.

Представь raycast как лазерный луч, с помощью которого можно определить, пересёк ли он какой-либо объект.

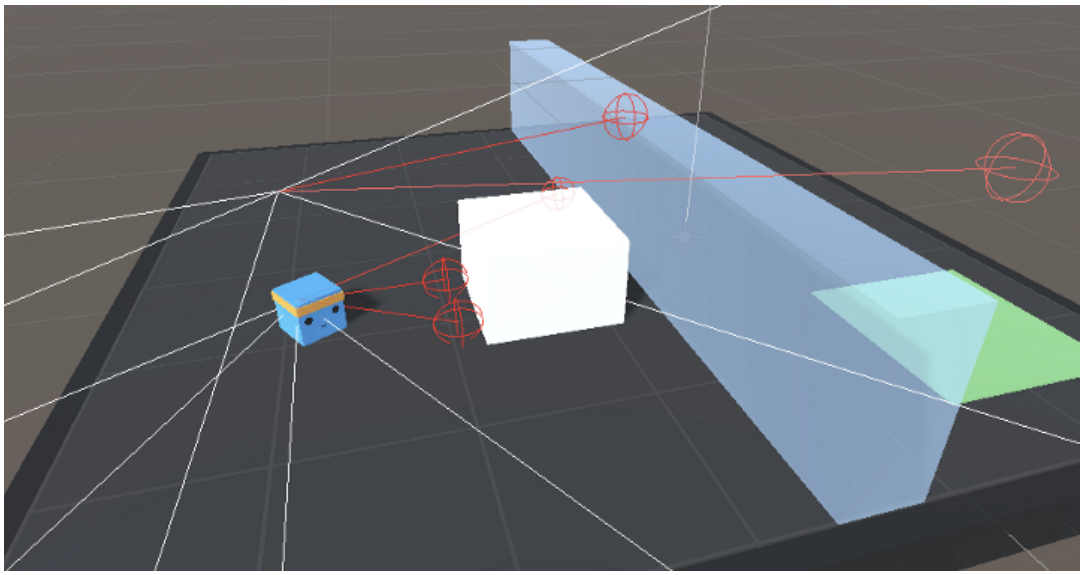


Рис. 13: Принцип трассировки лучей (источник: документация ML-Agents) [оригинал]

В этой среде агент использует несколько наборов лучей:

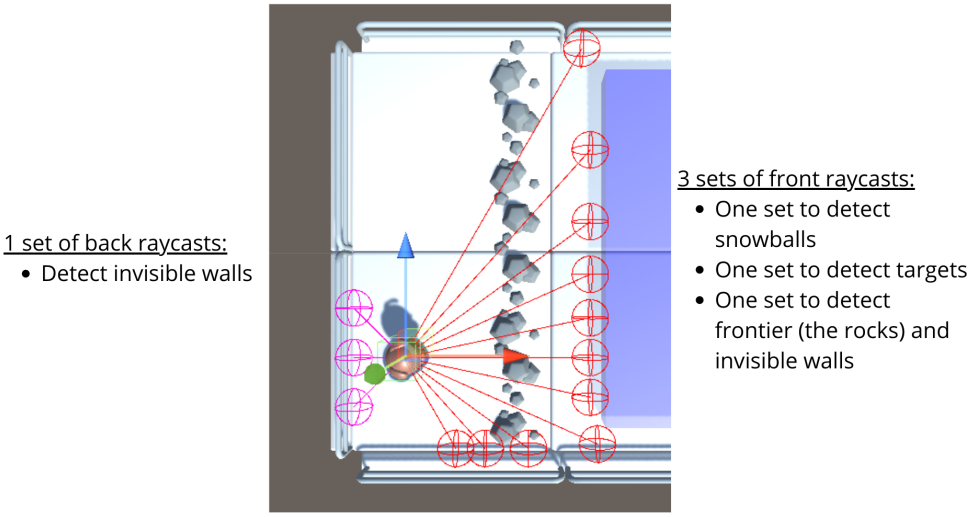


Рис. 14: Наблюдения агента SnowballTarget, получаемые с помощью raycast [оригинал]

Помимо лучей, агент получает логическое наблюдение «можно ли сейчас стрелять».

```
/// <summary>
/// Collect the observations (vector obs)
/// </summary>
/// <param name="sensor"></param>
8 références
public override void CollectObservations(VectorSensor sensor)
{
    // Can I shoot or not? (Bool)
    sensor.AddObservation(possibleShoot);
}
```

Рис. 15: Код сбора наблюдений в SnowballTarget [оригинал]

3.4 Пространство действий

Пространство действий дискретное:

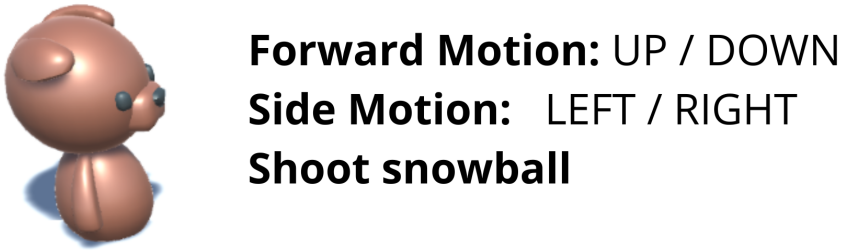


Рис. 16: Дискретное пространство действий SnowballTarget [оригинал]

4 Среда Pyramids

Цель обучения в этой среде — научить агента **добираться до золотого кирпича на вершине пирамиды. Для этого нужно нажать кнопку, чтобы появилась пирамида, подойти к ней, опрокинуть её и добраться до верхнего золотого кирпича**.

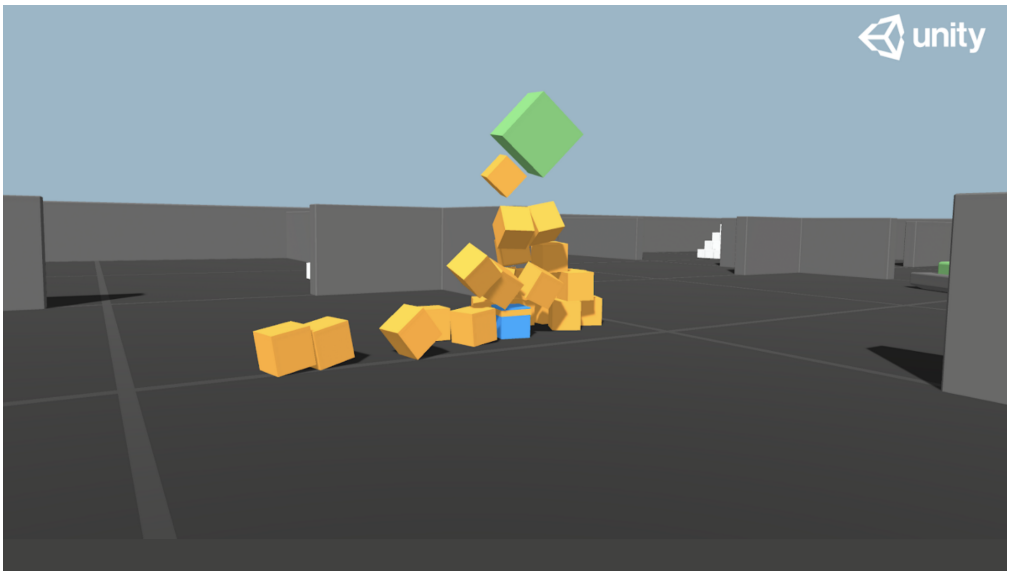


Рис. 17: Среда обучения Pyramids [оригинал]

4.1 Функция вознаграждения

Функция вознаграждения устроена следующим образом:

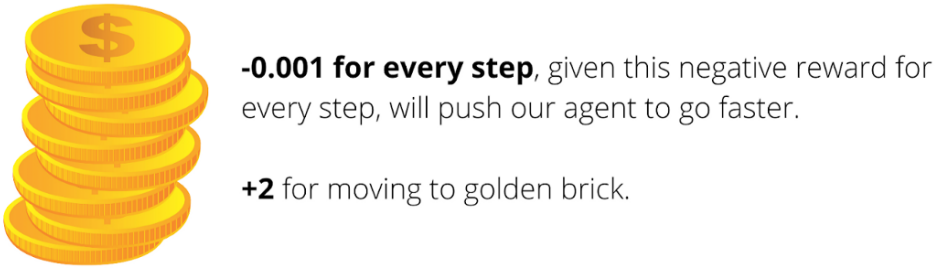


Рис. 18: Структура вознаграждения в среде Pyramids [оригинал]

В коде она выглядит так:

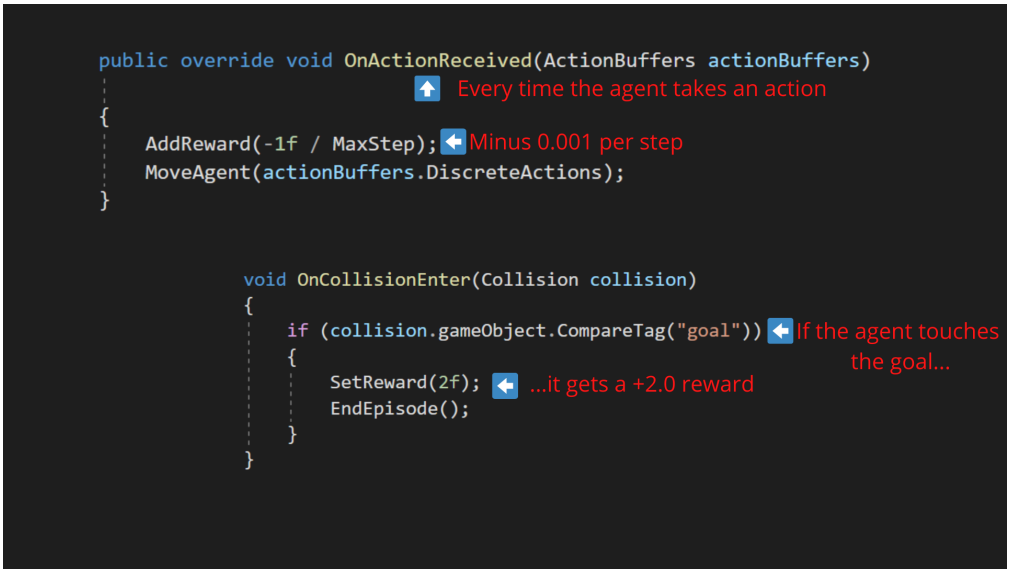


Рис. 19: Код вычисления вознаграждения в Pyramids [оригинал]

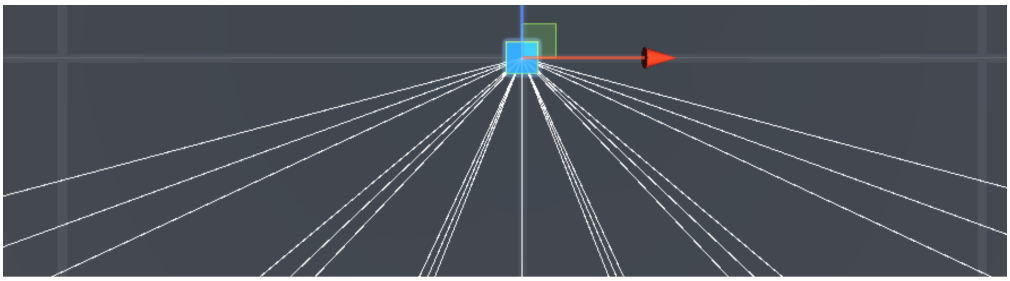
Чтобы научить нового агента сначала искать кнопку, а затем пирамиду, которую требуется разрушить, мы объединим два вида вознаграждения:

- *Внешнее (extrinsic)* вознаграждение от среды (показано на рисунке выше).
- *Внутреннее (intrinsic)* вознаграждение, называемое **любопытством (curiosity)**. Оно будет **побуждать агента проявлять любопытство, то есть лучше исследовать окружающую среду**.

Если хочется подробнее узнать о любопытстве, в следующем необязательном разделе объясняются основы этого подхода.

4.2 Пространство наблюдений

В качестве наблюдений мы **используем 148 лучей; каждый из них способен обнаруживать объекты** (переключатель, кирпичи, золотой кирпич и стены).



3 sets of front raycasts:

- All three detect:
  - Blocks
  - Walls
  - Goal (green cube)
  - Switch on
  - Switch off
  - Stone

Рис. 20: Наблюдения агента Pyramids с помощью трассировки лучей [оригинал]

Также используется **логическая переменная, показывающая состояние переключателя** (включён ли он и должна ли появиться пирамида), и вектор, **содержащий скорость агента**.



Рис. 21: Код сбора наблюдений в Pyramids [оригинал]

4.3 Пространство действий

Пространство действий **дискретное** и включает четыре возможных действия:

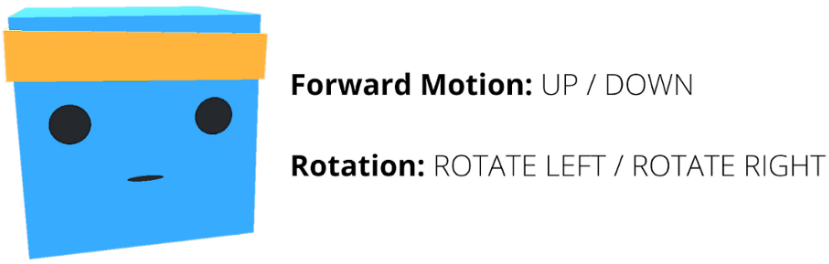


Рис. 22: Дискретное пространство действий Pyramids [оригинал]

5 (Необязательно) Что такое любопытство в глубоком обучении с подкреплением?

Это необязательное введение в метод любопытства (Curiosity). Чтобы узнать больше, можно прочитать две статьи с подробным математическим разбором:

- Обучение на основе любопытства через предсказание следующего состояния
- Дистилляция случайной сети: новый подход к обучению на основе любопытства

5.1 Две главные проблемы современного RL

Чтобы понять, что такое любопытство, сначала рассмотрим две основные проблемы обучения с подкреплением.

Первая — *проблема разреженных вознаграждений*: **большинство сигналов вознаграждения не несёт информации, поэтому равняется нулю**.

Вспомним, что RL опирается на *гипотезу вознаграждения*: любую цель можно описать как максимизацию вознаграждения. Поэтому вознаграждения служат обратной связью для агентов. **Если агент не получает такой обратной связи, он не может скорректировать представление о том, какие действия являются подходящими, а какие нет**.

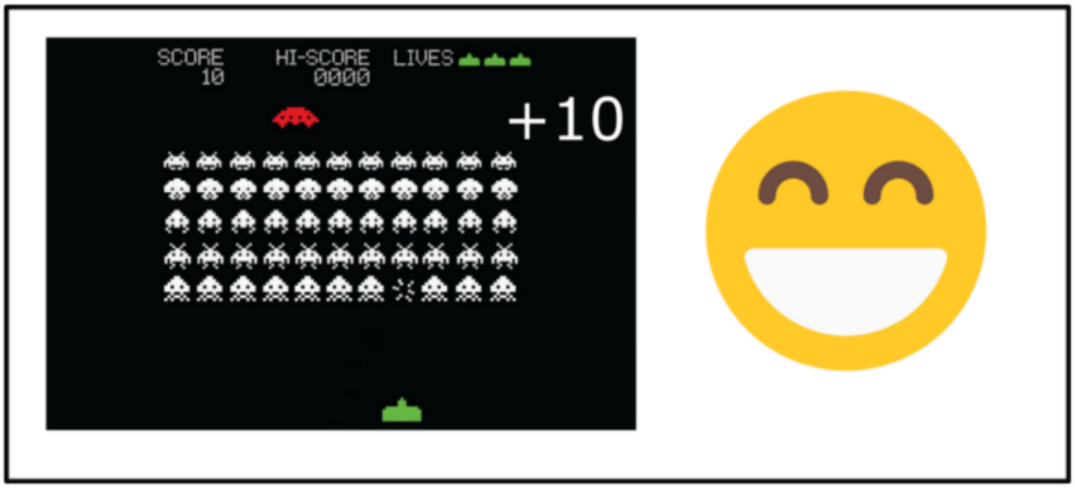


Рис. 23: Благодаря вознаграждению агент узнаёт, что действие в данном состоянии было хорошим [оригинал]

Например, в ViZDoom — наборе сред на основе игры Doom — в сценарии «DoomMyWayHome» агент получает вознаграждение **только за обнаружение бронежилета**. Однако жилет находится далеко от начальной точки, поэтому почти все награды будут нулевыми. Если агент не получает полезной обратной связи (плотных наград), поиск оптимальной политики займёт значительно больше времени: **агент может долго кружить на месте, так и не найдя цель**.

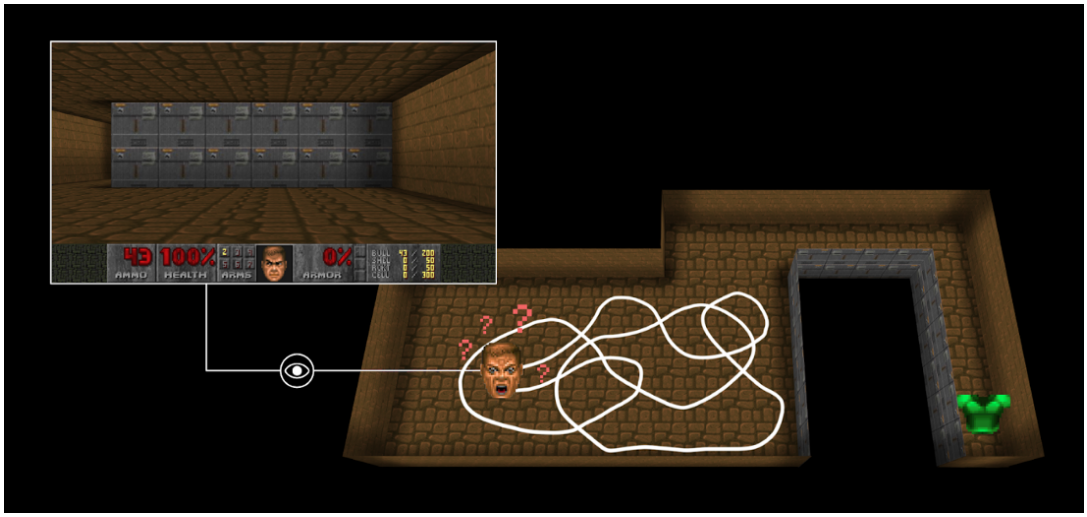


Рис. 24: Разреженные вознаграждения в среде DoomMyWayHome [оригинал]

Вторая большая проблема в том, что **внешнюю функцию вознаграждения создаёт человек: для каждой среды её нужно реализовывать вручную**. Но как масштабировать такой подход на большие и сложные среды?

5.2 Так что же такое любопытство?

Одно из решений состоит в том, чтобы **создать внутреннюю функцию вознаграждения, генерируемую самим агентом**. Тогда агент учится самостоятельно: одновременно выступает в роли ученика и источника собственной обратной связи.

Этот механизм внутреннего вознаграждения называется любопытством (Curiosity), потому что он побуждает агента исследовать новые и незнакомые состояния. Для этого агент получает большое вознаграждение при исследовании новых траекторий.

Этот механизм вдохновлён человеческим поведением: **нам от природы свойственно стремиться исследовать окружающий мир и открывать что-то новое**.

Внутреннее вознаграждение можно вычислять разными способами. Классический подход — любопытство на основе предсказания следующего состояния — заключается в том, чтобы измерять любопытство **как ошибку предсказания агентом следующего состояния по текущему состоянию и выбранному действию**.

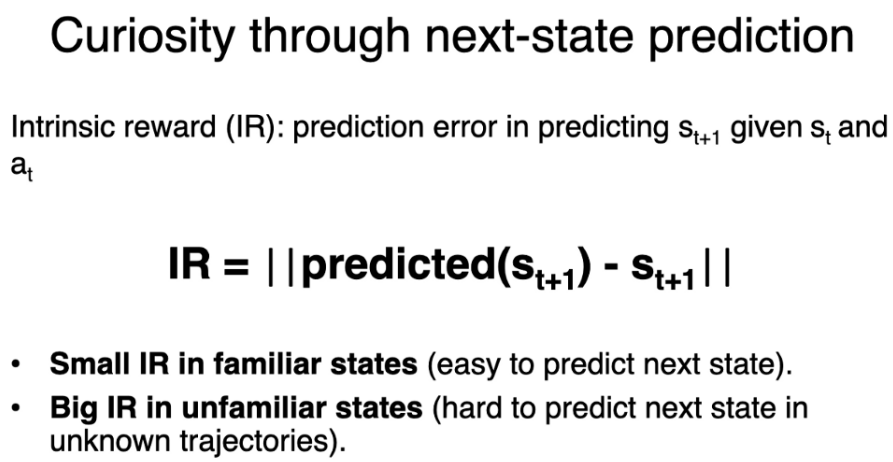


Рис. 25: Внутреннее вознаграждение за любопытство на основе ошибки предсказания следующего состояния [оригинал]

Идея любопытства состоит в том, чтобы **побуждать агента выполнять действия, уменьшающие неопределённость в его способности предсказывать последствия собственных действий**. Неопределённость выше в областях, где агент провёл мало времени, или там, где динамика сложна.

Если агент много раз посещает такие состояния, он начинает хорошо предсказывать следующее состояние (низкое любопытство). Если же он попадает в новое, неисследованное состояние, предсказание оказывается сложным (высокое любопытство).

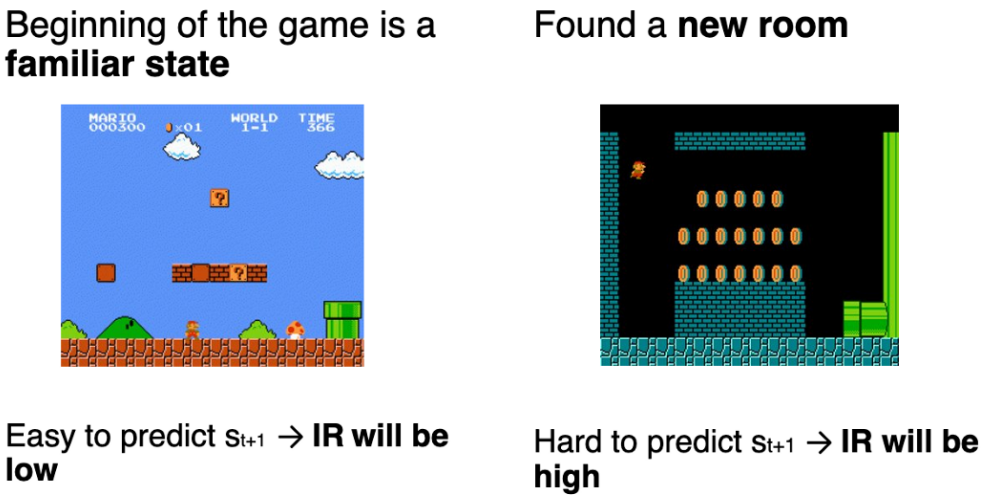


Рис. 26: Любопытство и исследование ранее неизвестных состояний [оригинал]

Использование любопытства побуждает агента отдавать предпочтение переходам с большой ошибкой предсказания (обычно это малоизученные области либо области со сложной динамикой) и **тем самым лучше исследовать среду**.

Существуют и **другие методы вычисления любопытства**. В ML-Agents используется более продвинутый подход — любопытство через дистилляцию случайной сети (Random Network Distillation). Его подробности выходят за рамки этого руководства, но при желании можно прочитать мою статью с детальным объяснением.

5.3 Практическая часть

Официальный практический ноутбук модуля 5 (Google Colab)

Помощь сообщества

Мы узнали, что представляет собой ML-Agents и как он работает, а также изучили две среды, которые будем использовать. Теперь можно обучать агентов!

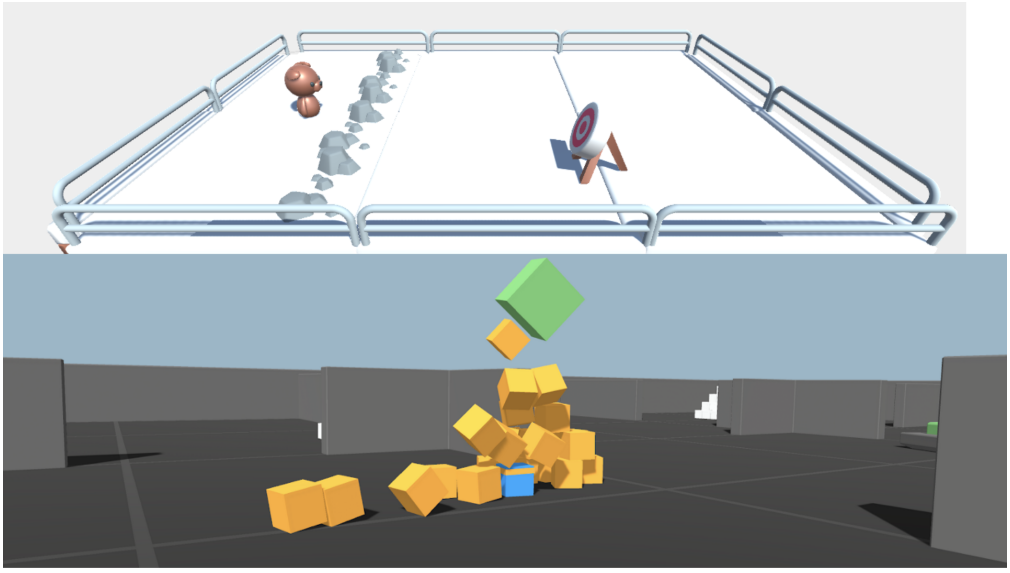


Рис. 27: Среды обучения SnowballTarget и Pyramids [оригинал]

Чтобы зачесть это практическое задание при получении сертификата, **достаточно загрузить обученные модели в Hub**. **Минимальных требований к результату нет**. Но для получения хороших результатов можно попробовать достичь следующих значений:

- Для Pyramids: среднее вознаграждение = 1,75.
- Для SnowballTarget: среднее вознаграждение = 15 либо 30 поражённых мишеней за эпизод.

Подробнее о сертификации смотри в https : / / huggingface . co / deep-rl-course / en / unit0 / introduction#certification-process

Чтобы начать практику, нажмите кнопку Open In Colab:

Открыть в Google Colab

Мы настоятельно **рекомендуем студентам выполнять практические задания в Google Colab**, а не на собственном компьютере.

Google Colab позволяет **сосредоточиться на изучении и экспериментах, не отвлекаясь на технические вопросы** установки и настройки сред.

5.4 Модуль 5: введение в ML-Agents



Рис. 28: Обложка модуля курса, посвящённого Unity ML-Agents [оригинал]

В этом ноутбуке ты познакомишься с ML-Agents и обучишь двух агентов.

- Первый научится **бросать снежки в появляющиеся мишени**.
- Второму нужно нажать кнопку для создания пирамиды, подойти к ней, опрокинуть её, **а затем добраться до золотого кирпича наверху**. Для этого придётся исследовать среду, используя метод любопытства (curiosity).

После этого ты сможешь **наблюдать за игрой агентов прямо в браузере**.

Подробнее о сертификации смотри в https : / / huggingface . co / deep-rl-course / en / unit0 / introduction#certification-process

Вот пример того, **чему ты научишься к концу этого модуля**.

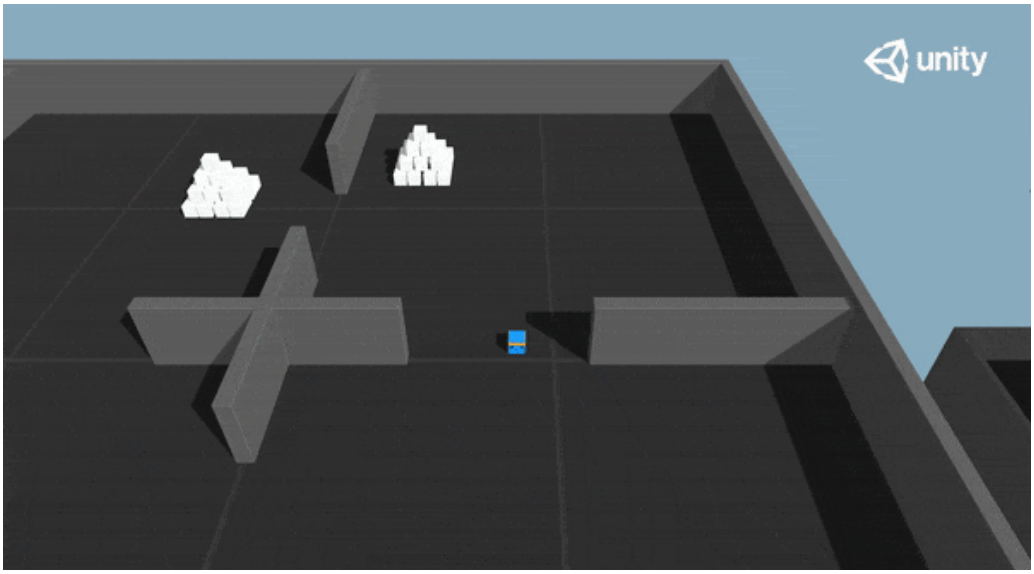


Рис. 29: Демонстрация агента Pyramids (кадр анимации) [оригинал]

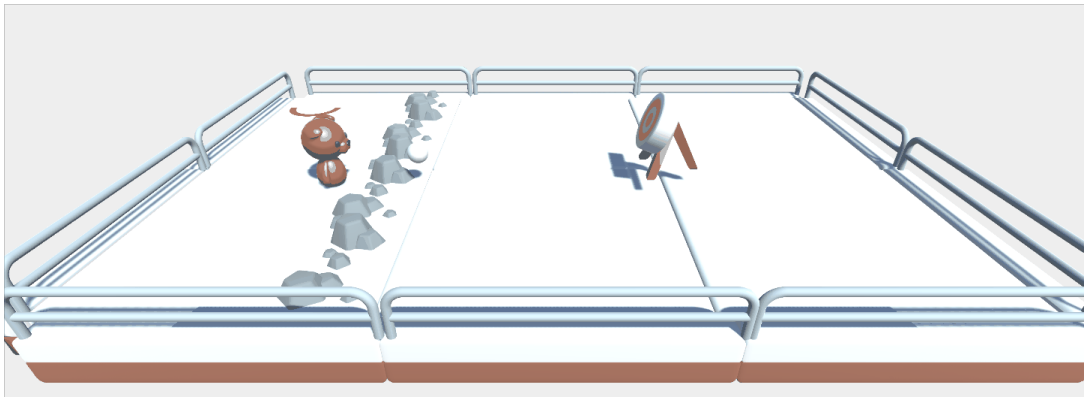


Рис. 30: Среда обучения SnowballTarget (кадр анимации) [оригинал]

Среды:

- Pyramids
- SnowballTarget

Библиотека RL:

- ML-Agents

Мы постоянно улучшаем свои руководства, поэтому, **если заметишь в этом ноутбуке проблему**, пожалуйста, создай обращение в репозитории GitHub.

Цели этого ноутбука

После прохождения ноутбука ты сможешь:

- Понимать принципы работы **ML-Agents** и библиотеки сред.
- **Обучать агентов в средах Unity**.

Предварительные знания

Прежде чем приступить к ноутбуку, необходимо:

Изучить устройство и принципы работы ML-Agents в теоретической части модуля 5.

5.5 Приступим к обучению агентов

Настройка GPU

- Чтобы **ускорить обучение агента, воспользуемся GPU**. Для этого выбери в Colab `Runtime > Change Runtime type`.

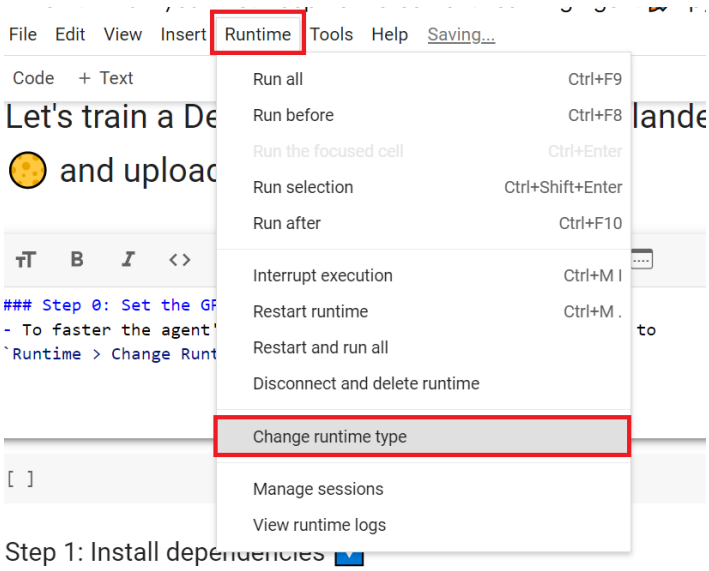


Рис. 31: Выбор типа среды выполнения в Google Colab [оригинал]

- `Hardware Accelerator > GPU`

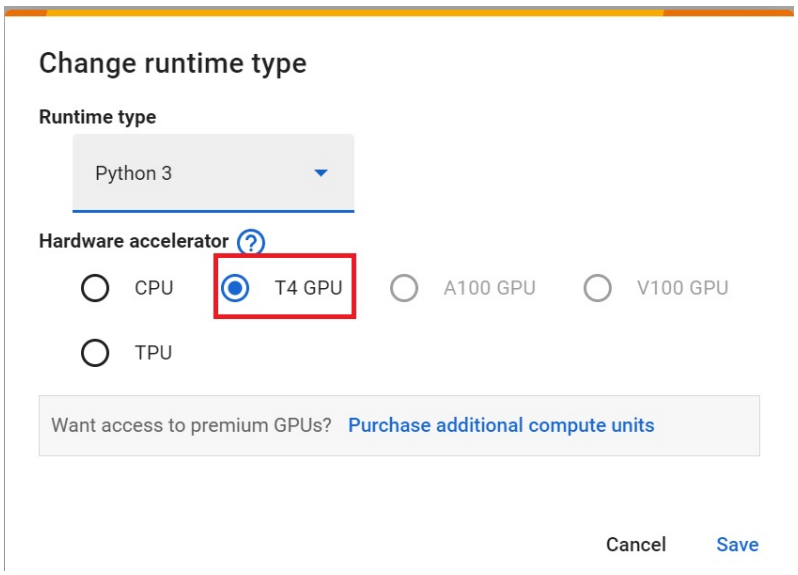


Рис. 32: Включение ускорителя GPU в Google Colab [оригинал]

Клонирование репозитория

- Необходимо клонировать репозиторий, содержащий **ML-Agents**.

```
# Clone the repository (can take 3min)
git clone --depth 1 https://github.com/Unity-Technologies/ml-agents
```

Настройка виртуального окружения

- Для корректной работы **ML-Agents** в Colab версия Python должна соответствовать требованиям библиотеки.
- Поддерживаемые версии Python указаны параметром `python_requires` в файлах `setup.py`, необходимых для установки **ML-Agents**. Они расположены здесь:
  - `/content/ml-agents/ml-agents/setup.py`
  - `/content/ml-agents/ml-agents-envs/setup.py`

- Текущая версия Python в Colab (её можно проверить командой `!python --version`) не соответствует значению `python_requires`, поэтому установка может незаметно завершиться с ошибкой. Позднее при выполнении команд могут появиться сообщения вроде:
  - `/bin/bash: line 1: mlagents-learn: command not found`
  - `/bin/bash: line 1: mlagents-push-to-hf: command not found`
- Чтобы решить проблему, создадим виртуальное окружение с версией Python, совместимой с библиотекой **ML-Agents**.

Примечание: *Для совместимости в будущем всегда проверяй параметр* ***`python_requires`*** *в установочных файлах и, если версия Python в Colab несовместима, выбирай в приведённом ниже сценарии максимальную поддерживаемую версию Python для виртуального окружения.*

```
# Colab's Current Python Version (Incompatible with ML-Agents)
!python --version
```

```
# Install virtualenv and create a virtual environment
!pip install virtualenv
!virtualenv myenv

# Download and install Miniconda
!wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
!chmod +x Miniconda3-latest-Linux-x86_64.sh
!./Miniconda3-latest-Linux-x86_64.sh -b -f -p /usr/local

# Activate Miniconda and install Python ver 3.10.12
!source /usr/local/bin/activate
!conda install -q -y --prefix /usr/local python=3.10.12 ujson  # Specify the version here

# Set environment variables for Python and conda paths
!export PYTHONPATH=/usr/local/lib/python3.10/site-packages/
!export CONDA_PREFIX=/usr/local/envs/myenv
```

```
# Python Version in New Virtual Environment (Compatible with ML-Agents)
!python --version
```

Установка зависимостей

```
# Go inside the repository and install the package (can take 3min)
%cd ml-agents
pip3 install -e ./ml-agents-envs
pip3 install -e ./ml-agents
```

SnowballTarget

Если нужно вспомнить, как устроена эта среда, смотри раздел https : / / huggingface . co / deep-rl-course/unit5/snowball-target

Загрузка и размещение ZIP-архива среды в `./training-envs-executables/linux/`

- Исполняемый файл нашей среды находится внутри ZIP-архива.
- Его нужно скачать и поместить в каталог `./training-envs-executables/linux/`.
- Мы используем исполняемый файл для Linux, поскольку работаем в Colab, где установлена Ubuntu (Linux).

```
# Here, we create training-envs-executables and linux
mkdir ./training-envs-executables
mkdir ./training-envs-executables/linux
```

Файл SnowballTarget.zip скачивается из https://github.com/huggingface/Snowball-Target с помощью `wget`.

```
wget "https://github.com/huggingface/Snowball-Target/raw/main/SnowballTarget.zip" -O ./training-
    envs-executables/linux/SnowballTarget.zip
```

Распакуем архив с исполняемым файлом.

```
unzip -d ./training-envs-executables/linux/ ./training-envs-executables/linux/SnowballTarget.zip
```

Убедись, что полученный файл доступен для выполнения.

```
chmod -R 755 ./training-envs-executables/linux/SnowballTarget
```

Создание конфигурационного файла SnowballTarget

- В ML-Agents **гиперпараметры обучения задаются в файлах config.yaml**.

Гиперпараметров много. Чтобы лучше разобраться в каждом, прочитай соответствующие пояснения в документации.

Нужно создать файл конфигурации `SnowballTarget.yaml` в каталоге `./content/ml-agents/config/ppo/`.

Мы дадим предварительный вариант конфигурации, который можно скопировать в `SnowballTarget.yaml`, **но его необходимо адаптировать**.

```
behaviors:
  SnowballTarget:
    trainer_type: ppo
    summary_freq: 10000
    keep_checkpoints: 10
    checkpoint_interval: 50000
    max_steps: 200000
    time_horizon: 64
    threaded: true
    hyperparameters:
      learning_rate: 0.0003
      learning_rate_schedule: linear
      batch_size: 128
      buffer_size: 2048
      beta: 0.005
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
    network_settings:
      normalize: false
      hidden_units: 256
      num_layers: 2
      vis_encode_type: simple
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
```

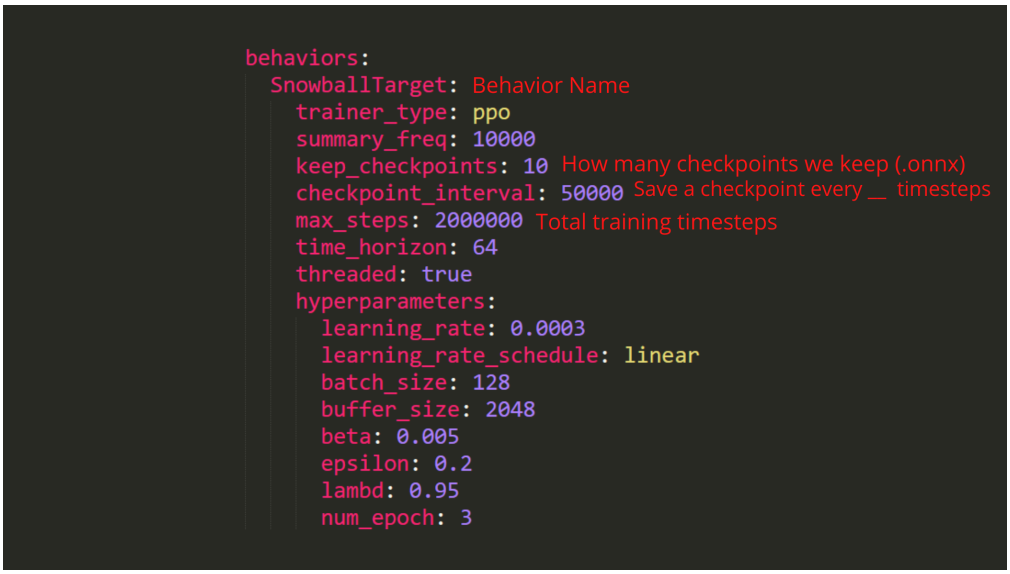


Рис. 33: Конфигурация обучения PPO для SnowballTarget, часть 1 [оригинал]



Рис. 34: Конфигурация обучения PPO для SnowballTarget, часть 2 [оригинал]

В качестве эксперимента попробуй изменить другие гиперпараметры. Unity предоставляет подробную документацию с объяснением каждого из них.

Теперь, когда конфигурационный файл создан и ты понимаешь назначение большинства гиперпараметров, можно обучать агента.

Обучение агента Для обучения нужно **запустить mlagents-learn и указать исполняемый файл среды**.

Задаются четыре параметра:

1. `mlagents-learn <config>` — путь к файлу конфигурации с гиперпараметрами.
2. `--env` — путь к исполняемому файлу среды.
3. `--run_id` — имя запуска обучения.
4. `--no-graphics` — не запускать визуализацию во время обучения.

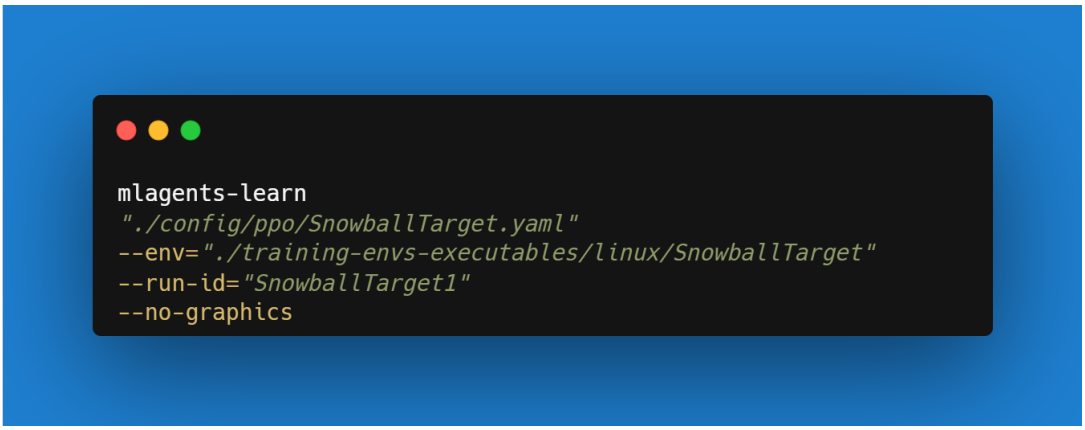


Рис. 35: Параметры команды обучения ML-Agents [оригинал]

Обучи модель; если обучение прервётся, для продолжения используй флаг `--resume`.

> При первом использовании `--resume` команда может завершиться ошибкой. Попробуй выполнить блок повторно, чтобы обойти её.

Обучение займёт от 10 до 35 минут в зависимости от конфигурации. Можно сходить за кофе — ты заслужил!

```
mlagents-learn ./config/ppo/SnowballTarget.yaml --env=./training-envs-executables/linux/
    SnowballTarget/SnowballTarget --run-id="SnowballTarget1" --no-graphics
```

Публикация агента в Hugging Face Hub

- После обучения мы **готовы загрузить агента в Hub и посмотреть на его поведение через браузер**.

Чтобы поделиться моделью с сообществом, осталось выполнить три шага:

1. Если аккаунта Hugging Face ещё нет, зарегистрируйся: https://huggingface.co/join
2. Войди в аккаунт и сохрани токен аутентификации, полученный на сайте Hugging Face.

- Создай новый токен (https://huggingface.co/settings/tokens) **с правом записи (write)**.

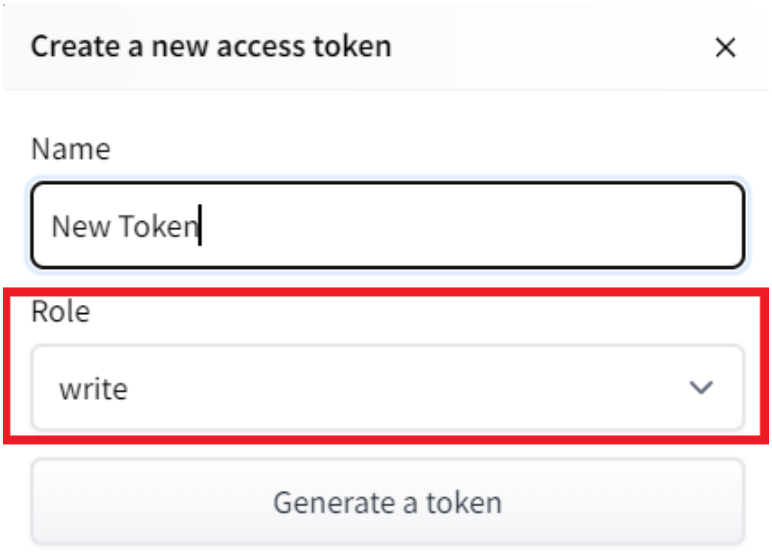


Рис. 36: Создание токена доступа Hugging Face [оригинал]

- Скопируй токен.
- Выполни ячейку ниже и вставь токен.

```
from huggingface_hub import notebook_login

notebook_login()
```

Если ты не используешь Google Colab или Jupyter Notebook, вместо этого выполни команду `huggingface-cli login`.

Затем нужно запустить `mlagents-push-to-hf`.

Для этого задаём четыре параметра:

1. `--run-id` — имя запуска обучения.
2. `--local-dir` — расположение сохранённого агента. Модели сохраняются в `results/`; например, `results/First Training`.
3. `--repo-id` — имя репозитория Hugging Face, который требуется создать или обновить. Его формат всегда такой: `<your huggingface username>/<the repo name>`. Если репозиторий ещё не существует, **он будет создан автоматически**.
4. `--commit-message` — сообщение коммита, поскольку репозитории Hugging Face основаны на Git.

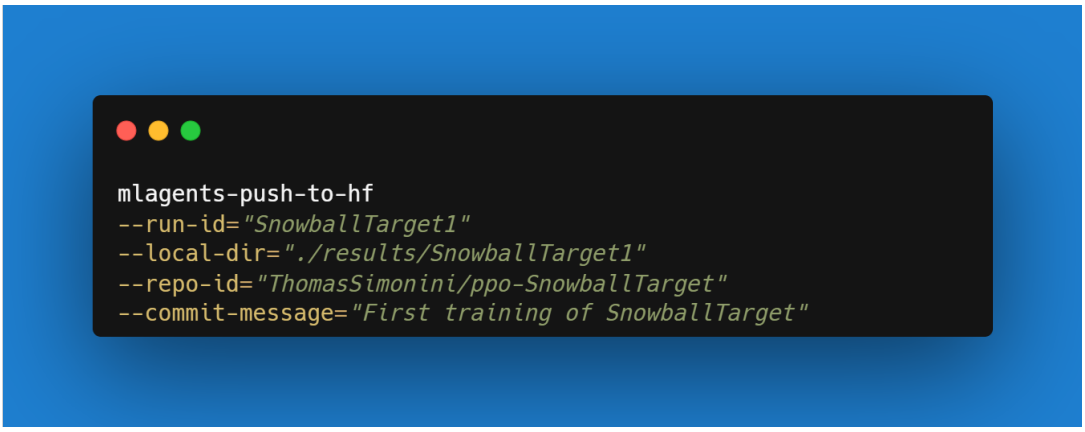


Рис. 37: Команда публикации модели ML-Agents в Hub [оригинал]

Например:

```
mlagents-push-to-hf  --run-id="SnowballTarget1" --local-dir="./results/SnowballTarget1" --repo-id="
    ThomasSimonini/ppo-SnowballTarget"  --commit-message="First Push"
```

```
mlagents-push-to-hf  --run-id= # Add your run id  --local-dir= # Your local dir  --repo-id= # Your
    repo id  --commit-message= # Your commit message
```

Если всё прошло успешно, в конце появится такое сообщение (но с другим URL):

```
Your model is pushed to the hub. You can view your model here: https://huggingface.co/
    ThomasSimonini/ppo-SnowballTarget
```

Это ссылка на твою модель. В репозитории есть карточка модели с инструкцией по её использованию, данные TensorBoard и конфигурационный файл. **Особенно удобно, что это Git-репозиторий: можно хранить разные коммиты, обновлять модель новыми публикациями и т. д.**

Но самое интересное впереди: **возможность посмотреть на работу агента онлайн**.

Посмотрим, как играет агент Этот шаг очень простой:

1. Открой демонстрацию SnowballTarget в Hugging Face Spaces.
2. Запусти игру и разверни её на весь экран кнопкой в правом нижнем углу.

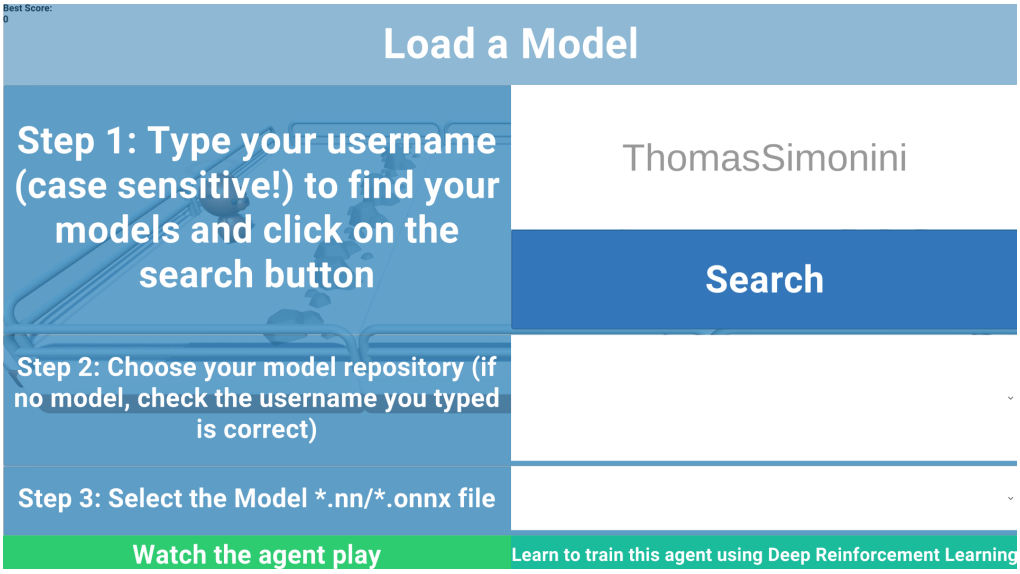


Рис. 38: Загрузка обученного агента SnowballTarget в Hugging Face Space [оригинал]

1. На первом шаге введи имя пользователя. Регистр важен: например, моё имя — ThomasSimonini, а не thomassimonini или ThOmasImoNInI. Затем нажми кнопку поиска.
2. На втором шаге выбери репозиторий модели.
3. На третьем шаге **выбери, какую именно модель нужно воспроизвести**:

- У меня их несколько, поскольку мы сохраняли модель через каждые 500000 временных шагов.
- Но поскольку я хочу посмотреть самую новую, выбираю `SnowballTarget.onnx`.

Полезно **попробовать модели с разных этапов обучения, чтобы увидеть прогресс агента**.

И не стесняйся делиться лучшим результатом агента в Discord, в канале `#rl-i-made-this`.

Теперь попробуем более сложную среду под названием Pyramids.

Pyramids

Загрузка и размещение ZIP-архива среды в `./training-envs-executables/linux/`

- Исполняемый файл среды упакован в ZIP-архив.
- Скачай его и помести в каталог `./training-envs-executables/linux/`.
- Используется исполняемый файл Linux, поскольку мы работаем в Colab, где установлена Ubuntu (Linux).

Файл Pyramids.zip скачиваем из https://huggingface.co/spaces/unity/ML-Agents-Pyramids/resolve/main/Pyramids.zip с помощью `wget`.

```
wget "https://huggingface.co/spaces/unity/ML-Agents-Pyramids/resolve/main/Pyramids.zip" -O ./
    training-envs-executables/linux/Pyramids.zip
```

Распакуй его.

```
unzip -d ./training-envs-executables/linux/ ./training-envs-executables/linux/Pyramids.zip
```

Убедись, что файл доступен для выполнения.

```
chmod -R 755 ./training-envs-executables/linux/Pyramids/Pyramids
```

Изменение конфигурационного файла PyramidsRND

- В отличие от первой, созданной нами среды, **Pyramids разработана командой Unity**.
- Поэтому конфигурационный файл PyramidsRND уже существует и находится здесь: `./content/ml-agents/config/ppo/PyramidsRND.yaml`.
- Возможно, ты задаёшься вопросом, почему в названии PyramidsRND есть «RND». Эта аббревиатура означает *Random Network Distillation* (дистилляция случайной сети) — способ генерировать вознаграждение за любопытство. Если хочется узнать больше, мы подготовили статью с подробным объяснением этого метода.

Для этого обучения мы изменим один параметр:

- Общее число шагов обучения задано слишком большим: требуемого результата (среднее вознаграждение = 1,75) можно достичь всего за 1 млн шагов. Поэтому откроем `config/ppo/PyramidsRND.yaml` и **изменим `max_steps` на 1000000**.

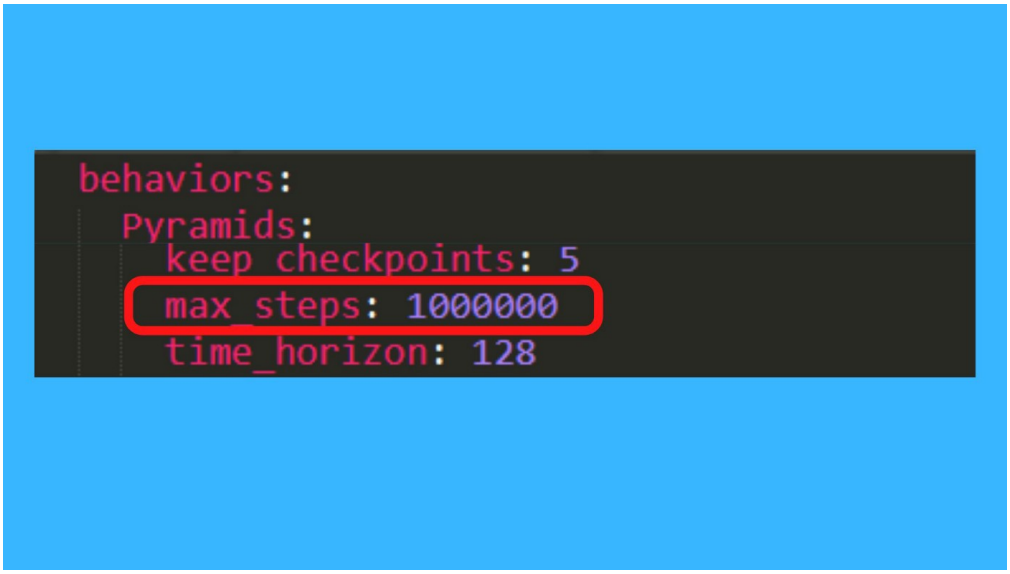


Рис. 39: Конфигурация PyramidsRND и максимальное количество шагов обучения [оригинал]

В качестве эксперимента попробуй изменить другие гиперпараметры. Unity предоставляет хорошую документацию с объяснением каждого из них.

Теперь можно приступать к обучению агента.

Обучение агента Обучение займёт от 30 до 45 минут в зависимости от компьютера. Можешь сходить за кофе — ты заслужил!

```
mlagents-learn ./config/ppo/PyramidsRND.yaml --env=./training-envs-executables/linux/Pyramids/
    Pyramids --run-id="Pyramids Training" --no-graphics
```

Публикация агента в Hugging Face Hub

- Теперь, когда агент обучен, мы **готовы загрузить его в Hub, чтобы затем посмотреть на его поведение в браузере**.

```
mlagents-push-to-hf  --run-id= # Add your run id  --local-dir= # Your local dir  --repo-id= # Your
    repo id  --commit-message= # Your commit message
```

Посмотрим, как играет агент https://huggingface.co/spaces/unity/ML-Agents-Pyramids

Бонус: почему бы не обучить агента в другой среде? Теперь, когда ты умеешь обучать агента с помощью ML-Agents, **почему бы не попробовать другую среду?**

В ML-Agents есть 17 разных сред, и мы также создаём собственные. Лучший способ научиться — экспериментировать самостоятельно и получать удовольствие от процесса.

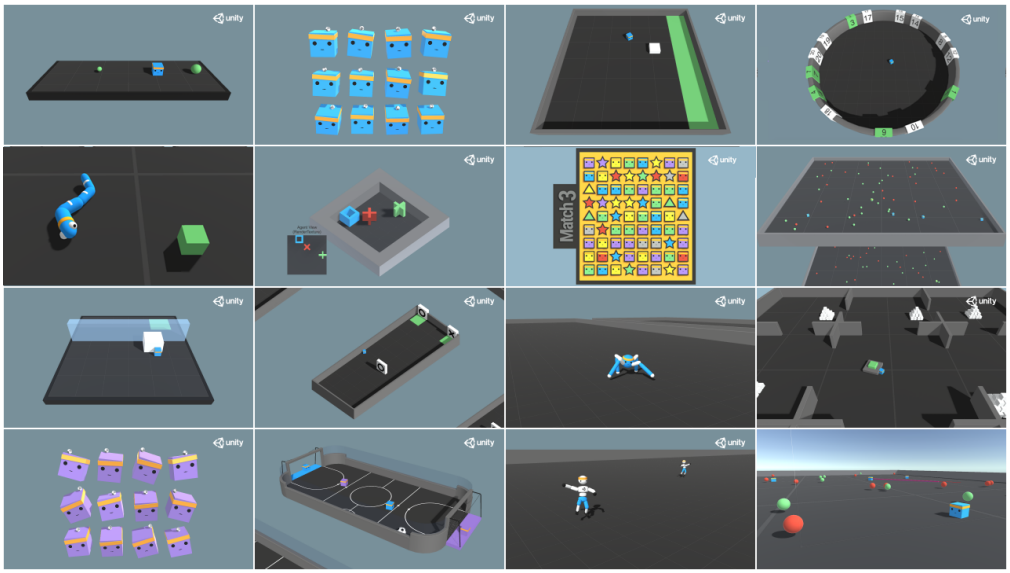


Рис. 40: Пример другой среды Unity ML-Agents [оригинал]

Полный список доступных сейчас на Hugging Face сред находится здесь: https://github.com/huggingface/ml-agents#the-environments

Демонстрации, в которых можно наблюдать за своими агентами: https://huggingface.co/unity

На данный момент мы интегрировали:

- Демонстрацию Worm, в которой нужно научить **червя ползать**.
- Демонстрацию Walker, в которой агент учится **идти к цели**.

На сегодня всё. Поздравляем с завершением практического занятия!

Лучший способ учиться — практиковаться и экспериментировать. Почему бы не попробовать ещё одну среду? В ML-Agents есть 18 разных сред, но можно создавать и собственные. Изучай документацию и получай удовольствие!

Увидимся в модуле 6!

Продолжай учиться и не теряй энтузиазма

6 Проверочный тест

Лучший способ учиться и избежать иллюзии компетентности — **проверять себя**. Это поможет понять, **какие знания ещё нужно закрепить**.

Вопрос 1. Какие из следующих инструментов специально предназначены для разработки видеоигр?

- **A.** Unity (C#)
- **B.** Unreal Engine (C++)
- **C.** Godot (GDScript, C++, C#)
- **D.** JetBrains Rider
- **E.** JetBrains CLion
- **F.** Microsoft Visual Studio и Visual Studio Code

Вопрос 2. Какие из следующих утверждений об Unity ML-Agents верны?

- **A.** Объекты типа Scene в Unity можно использовать для создания сред обучения.
- **B.** Unity ML-Agents позволяет создавать агентов и обучать их с помощью обучения с подкреплением.
- **C.** Компонент `Communicator` отвечает за связь между средами и агентами Unity на C# и серверной частью на Python.
- **D.** В процессе обучения используются алгоритмы обучения с подкреплением, реализованные на PyTorch.
- **E.** Unity ML-Agents поддерживает только Proximal Policy Optimization (PPO).
- **F.** Включает обёртку Gym и её мультиагентный аналог `PettingZoo`.

Вопрос 3. Вставь пропущенные буквы

- В Unity ML-Agents политика агента называется b _ _ _ n.
- Компонент, который координирует работу агентов, называется _ c _ _ _ m _.

Вопрос 4. Объясни своими словами, что такое `raycast`

Вопрос 5. Чем отличается получение наблюдений через `frames` и через `raycasts`?

- **A.** При использовании `frames` среда описывается всеми пикселями изображения, а при использовании `raycasts` передаётся только выборка этих пикселей.
- **B.** При использовании `raycasts` среда описывается пикселями изображения, а при использовании `frames` создаётся (обычно) линия для проверки столкновений с объектами.
- **C.** При использовании `frames` собираются все пиксели изображения, описывающего среду; при использовании `raycast` пиксели не нужны: обычно создаются лучи и проверяются их пересечения с объектами.

Вопрос 6. Назови несколько входных переменных среды и агента, используемых для обучения в SnowballTarget или Pyramids

Поздравляем с прохождением теста! Если какой-то материал оказался забытым, вернись к соответствующим разделам и закрепи знания.

6.1 Ответы на тест и пояснения

Решение вопроса 1

Правильные ответы: A, B, C

A. Unity (C#)

B. Unreal Engine (C++)

C. Godot (GDScript, C++, C#)

D. JetBrains Rider. Пояснение: хотя эта IDE полезна благодаря поддержке C# для Unity, она не предназначена специально для разработки видеоигр.

E. JetBrains CLion. Пояснение: хотя эта IDE полезна благодаря поддержке C++ для Unreal Engine, она не специализирована на разработке видеоигр.

F. Microsoft Visual Studio и Visual Studio Code. Пояснение: эти универсальные среды разработки поддерживают в том числе Unity и Unreal Engine, но не ориентированы исключительно на видеоигры.

Решение вопроса 2

Правильные ответы: A, B, C, D, F

A. Объекты типа Scene в Unity можно использовать для создания сред обучения.

B. Unity ML-Agents позволяет создавать агентов и обучать их с помощью обучения с подкреплением.

C. Компонент `Communicator` управляет обменом между средами и агентами Unity на C# и серверной частью на Python.

D. В процессе обучения используются алгоритмы обучения с подкреплением, реализованные на PyTorch.

E. Unity ML-Agents поддерживает только Proximal Policy Optimization (PPO). Пояснение: нет, Unity ML-Agents поддерживает несколько семейств алгоритмов, включая Actor-Critic, который будет разобран в следующем разделе.

F. Включает обёртку Gym и её мультиагентный аналог `PettingZoo`.

Решение вопроса 3

- b r a i n
- a c a d e m y

Решение вопроса 4

Raycast — это (в большинстве случаев) линейная проекция, подобная `лазерному лучу`, предназначенная для определения столкновений луча с объектами.

Решение вопроса 5

Правильный ответ: C

A. При использовании `frames` среда описывается всеми пикселями изображения, а при использовании `raycasts` передаётся лишь выборка этих пикселей. Пояснение: `raycasts` никак не связаны с пикселями. Это линейные проекции (лучи), которые проверяют пересечения с объектами.

B. При использовании `raycasts` среда описывается пикселями изображения, а при использовании `frames` создаётся линия для проверки пересечений с объектами. Пояснение: всё наоборот — `frames` собирают пиксели, а `raycasts` проверяют пересечения.

C. При использовании `frames` мы собираем все пиксели изображения, определяющего среду. При использовании `raycast` пиксели не нужны: обычно создаются лучи и проверяется их пересечение с объектами.

Решение вопроса 6

- Пересечения испущенных агентом лучей с блоками, (невидимыми) стенами, камнями, мишенью, переключателями и другими объектами.
- Обычные входные переменные, описывающие свойства агента, например его скорость.
- Логические переменные, например положение переключателя (включён/выключен) в Pyramids или `can I shoot?` («могу ли я стрелять?») в SnowballTarget.

7 Заключение

Поздравляем с завершением модуля! Ты только что обучил своих первых агентов с помощью ML-Agents и опубликовал их в Hub.

Лучший способ учиться — **практиковаться и экспериментировать**. Почему бы не попробовать другую среду? В ML-Agents есть 18 различных сред.

Например:

- Worm, где нужно научить червя ползать.
- Walker, где агент учится идти к цели.

Посмотри документацию, чтобы узнать, как обучать агентов в этих средах, а также найти список уже интегрированных в Hub сред ML-Agents: https://github.com/huggingface/ml-agents#getting-started

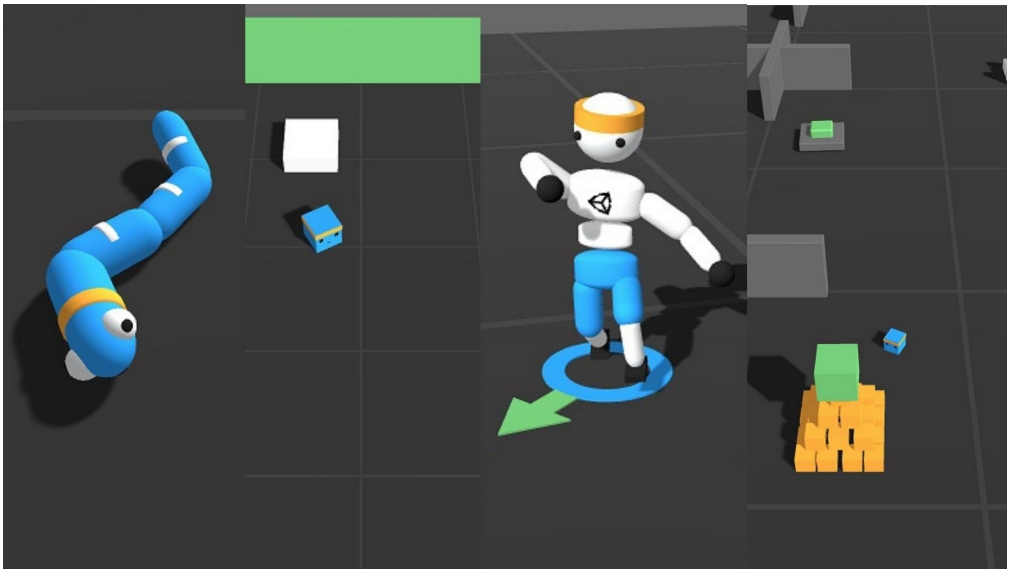


Рис. 41: Другие примеры сред Unity ML-Agents [оригинал]

В следующем модуле мы познакомимся с мультиагентными системами. Ты обучишь своих первых мультиагентных агентов, которые будут соревноваться с агентами других участников в футболе и битве снежками.

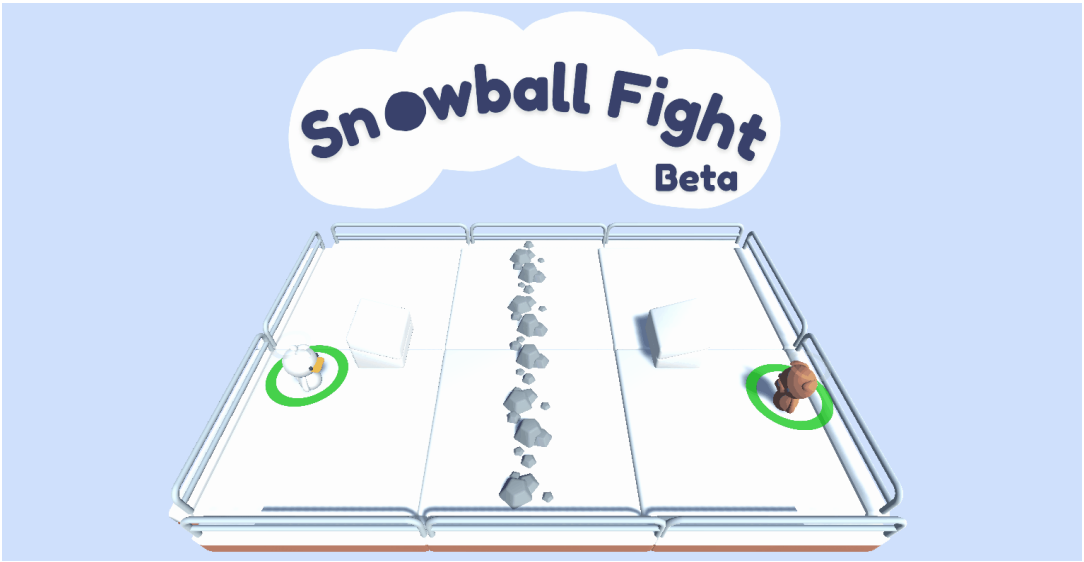


Рис. 42: Битва снежками между обученными агентами (кадр анимации) [оригинал]

Наконец, нам очень хочется **узнать твоё мнение о курсе и о том, как его можно улучшить**. Если есть идеи или замечания, пожалуйста, заполни эту форму.

Продолжай учиться и не теряй энтузиазма

8 Бонус: создание собственных сред с Unity и ML-Agents

С помощью Unity и ML-Agents можно создавать собственные среды обучения с подкреплением. На первый взгляд игровой движок Unity может показаться сложным, но следующие шаги

помогут освоить его постепенно.

8.1 Шаг 1. Научись пользоваться Unity

- Лучший способ освоить Unity — пройти курс «Create with Code». Это серия видеоуроков для начинающих, в которой **ты создашь пять небольших игр на Unity**.

8.2 Шаг 2. Создай простейшую среду по руководству

- Когда освоишь Unity, сможешь создать свою первую простую RL-среду с помощью этого руководства.

8.3 Шаг 3. Усложняй и совершенствуй среды

- После создания первой простой среды можно переходить к более сложным, используя документацию ML-Agents, особенно разделы Designing Agents и Agent.
- Кроме того, можно пройти бесплатный курс «Создание среды с колибри» от Адама Келли (Adam Kelly).

Удачи и приятных экспериментов! Если создашь собственную среду, обязательно поделись ею в Discord-канале `#rl-i-made-this`.